

Reporte

TAD

TAD ListaDesordenada

Datos:

- cabeza : Nodo
- tamaño : int

Operaciones:

- insertar(elemento: T) : void
 - buscar(elemento: T) : T
 - eliminar(elemento: T) : bool
 - estaVacia() : bool
 - size() : int
-

TAD ListaOrdenada

Datos:

- cabeza : Nodo
- tamaño : int

Operaciones:

- insertar(elemento: T) : void
 - buscar(elemento: T) : T
 - eliminar(elemento: T) : bool
 - estaVacia() : bool
 - size() : int
-

TAD TablaHash

Datos:

- tabla : arreglo de entradas
- tamaño : int

Operaciones:

- insertar(clave: K, valor: V) : void
 - buscar(clave: K) : V
 - eliminar(clave: K) : bool
 - contiene(clave: K) : bool
 - size() : int
-

TAD ArbolAVL

Datos:

- raiz : NodoAVL

Operaciones:

- insertar(elemento: T) : void
 - buscar(elemento: T) : T
 - eliminar(elemento: T) : bool
 - estaVacio() : bool
 - altura() : int
-

TAD ArbolB

Datos:

- raiz : NodoB
- orden : int

Operaciones:

- insertar(elemento: T) : void
 - buscar(elemento: T) : T
 - eliminar(elemento: T) : bool
 - estaVacio() : bool
-

TAD ArbolBMas

Datos:

- raiz : NodoBMas
- orden : int

Operaciones:

- insertar(elemento: T) : void
 - buscar(elemento: T) : T
 - eliminar(elemento: T) : bool
 - estaVacio() : bool
 - rango(inicio: T, fin: T) : iterable
-

TAD Grafo

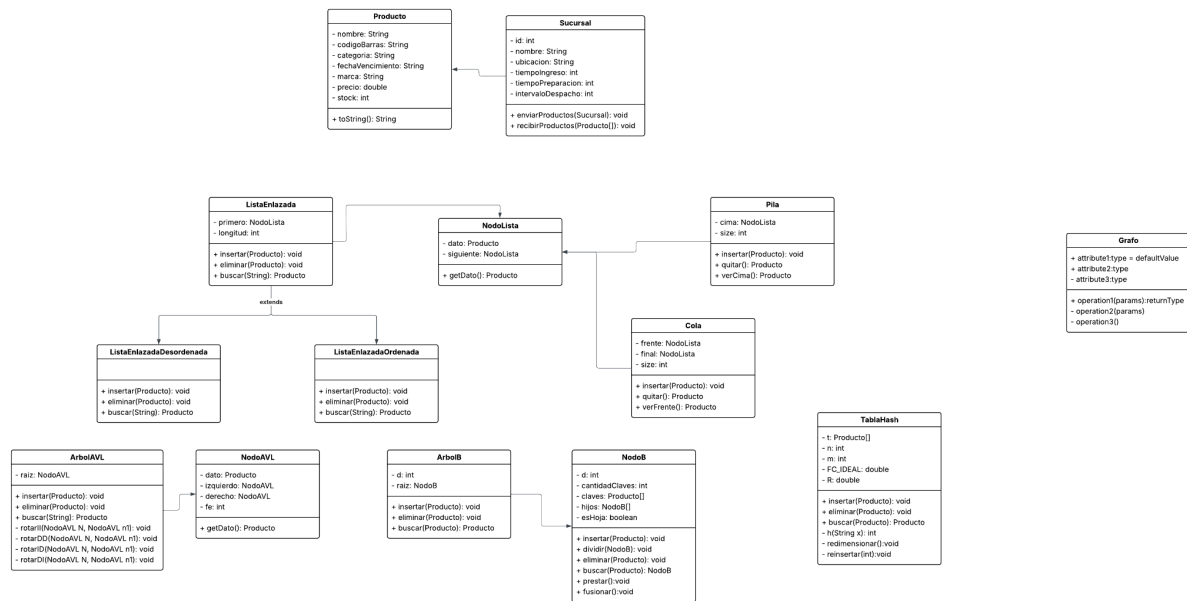
Datos:

- vertices : tabla hash
- adyacencia : tabla hash

Operaciones:

- insertarVertice(id: K) : void
- insertarArista(origen: K, destino: K, peso: double) : void
- eliminarVertice(id: K) : bool
- eliminarArista(origen: K, destino: K) : bool
- existeVertice(id: K) : bool
- existeArista(origen: K, destino: K) : bool

Clases



Complejidad Big-O

Sobre complejidad Big-O, en la lista desordenada la inserción es $O(1)$ porque se agrega al inicio; la búsqueda es $O(n)$ y la eliminación es $O(n)$ porque requieren recorrido lineal por código de barras.

En la lista ordenada la inserción es $O(n)$ por búsqueda de posición, la búsqueda es $O(n)$ y la eliminación es $O(n)$, ya que siguen dependiendo de recorrer la lista.

En AVL, búsqueda, inserción y eliminación son $O(\log n)$ gracias al balanceo por rotaciones y a la altura acotada.

En el árbol B, inserción y eliminación son $O(\log_m n)$, y la búsqueda puntual por clave también es logarítmica en términos de altura. Sin embargo, la búsqueda pública implementada por rango de fechas puede acercarse a $O(n)$ en peor caso, según el tamaño del rango consultado.

En el árbol B+, la búsqueda por categoría es $O(\log_d n)$, la inserción es $O(\log_d n)$, y la eliminación en la implementación actual puede degradar a $O(n)$ porque la localización por código de barras se hace con un recorrido desde la hoja más a la izquierda.

En la tabla hash todas las operaciones son $O(1)$ debido al acceso por arreglos